

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#_05_)

Use of Venus Simulator for Register Level Execution of RISC-V Control Instructions

Student Name: _Eesha Rehan_____ Roll No: 24i-6027_____ Section: B__

Date performed: _16th September _____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In the task1 part1 there are two tables table 1 is branch table and table 2 is jumps. Actual instructions map 1:1 to hardware opcodes, whereas pseudo-instructions are expanded by the assembler into equivalent base instructions. In task1part2 we are required code by using two instructions beq and j. beq is used here if the two operands are equal so it jumps to the label true and j jump to the end label means end of the code. In task1 part3 there are two codes given the further Explanation is in the comments.

Code:

Task1(2):

```
.data

.text

addi s0, x0, 5

addi s1, x0, 5

.globl main

beq s0,s1,true

j end

true:

addi s2, x0, 1

end:
```

```
1 .data
2 .text
3 addi s0, x0, 5
4 addi s1, x0, 5
5 .globl main
6
7 beq s0,s1,true
8 j end
9 true:
10 addi s2, x0, 1
11
12 end:
13
14 |
15
```

Task1 part3:

Code:1

.data

.text

.globl main

addi s0, s0, 4 #s0=s0+4 adding immediate 4 into s0

addi s1, s1, 6 #s1=s1+6 adding immediate 6 into s1

beq s0, s1, Fin #checks if s0==s1 and Fin is the target if it is equal

#it will move to Fin label

True: #label that is check when the condition (s0==s1) is not true

add s2, s1, s0 #adding 6+4 and storing it to s2 register

j end #unconditional jump that will move directly to end because its the target

Fin:# if condition s0==s1 it will execute this instruction

sub s2, s1, s0 #s2=s1-s0=6-4=2(subtracting)

end:#here the conditon end exeute

```
1 .data
2 .text
3
4 .globl main
5
6     addi s0, s0, 4    #s0=s0+4 adding immediate 4 into s0
7     addi s1, s1, 6    #s1=s1+6 adding immediate 6 into s1
8     beq  s0, s1, Fin  #checks if s0==s1 and Fin is the target if it is equal
9                     #it will move to Fin label
10 True:  #label that is check when the condition (s0==s1) is not true
11     add  s2, s1, s0   #adding 6+4 and storing it to s2 register
12     j    end         #unconditional jump that will move directly to end because its the target
13 Fin:# if condition s0==s1 it will execute this instruction
14     sub  s2, s1, s0   #s2=s1-s0=6-4=2(subtracting)
15 end:#here the conditon end exeute
16
```

Code2:

```
1 .data
2
3 .text
4 .globl main
5
6     addi s0, s0, 4#using addi to add const 4 into s0 and storing in s0
7     addi s1, s1, 4#using addi to add const 4 into s1 and storing in s1
8     beq  s0, s1, Fin #chceking if s0==s1 so jumpto label Fin
9 True:#true label if above condition not statisfied
10     add  s2, s1, s0  #adding s0 to s1 and stroing in s2
11     j    end#then jump to end
12 Fin:
13     sub  s2, s1, s0#if beq condition true so sub s1 and s0 and store resultin s2
14 End:#end of the code
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00500413	addi x8 x0 5	addi s0, x0, 5
0x4	0x00500493	addi x9 x0 5	addi s1, x0, 5
0x8	0x00940463	beq x8 x9 8	beq s0,s1,true
0xc	0x0080006F	jal x0 8	j end
0x10	0x00100913	addi x18 x0 1	addi s2, x0, 1

s0 (x8)	5	s1 (x9)	5
------------	---	------------	---

s2 (x18)	1
-------------	---

PC	Machine Code	Basic Code	Original Code
0x0	0x00440413	addi x8 x8 4	addi s0, s0, 4 #s0=s0+4 adding immediate 4 into s0
0x4	0x00648493	addi x9 x9 6	addi s1, s1, 6 #s1=s1+6 adding immediate 6 into s1
0x8	0x00940663	beq x8 x9 12	beq s0, s1, Fin #checks if s0==s1

```
0xc    0x00848933    add x18 x9    add s2, s1, s0
                                x8                #adding 6+4 and
                                                storing it to s2
                                                register
```

```
0x10    0x0080006F    jal x0 8      j end
                                                #unconditional
                                                jump that will
                                                move directly to
                                                end because its
                                                the target
```

```
0x14    0x40848933    sub x18 x9    sub s2, s1, s0
                                x8                #s2=s1-s0=6-
                                                4=2 (subtracting)
```

```
s2      10
(x18)
```

Task 2:

Description:

In this task code is given in c language we have to convert it in assembly language so we load values in the registers then checks the condition if branch is equal if a is equal to b so we move to the label true and perform the instruction of subtraction otherwise move to the label else where we do addition. And then end the code.

Code:

```
.data
```

```
.text
```

```
main:
```

```
    li s0, 5    #a=5
```

```
li s1, 10    #b=10
```

```
li s3, 4     #d=4
```

```
li s4, 3     #e=3
```

```
beq s0, s1, true
```

```
else:
```

```
add s2, s3, s4 #c=d+e
```

```
j end
```

```
true:
```

```
sub s2, s3, s4 #c=d-e
```

```
end:
```

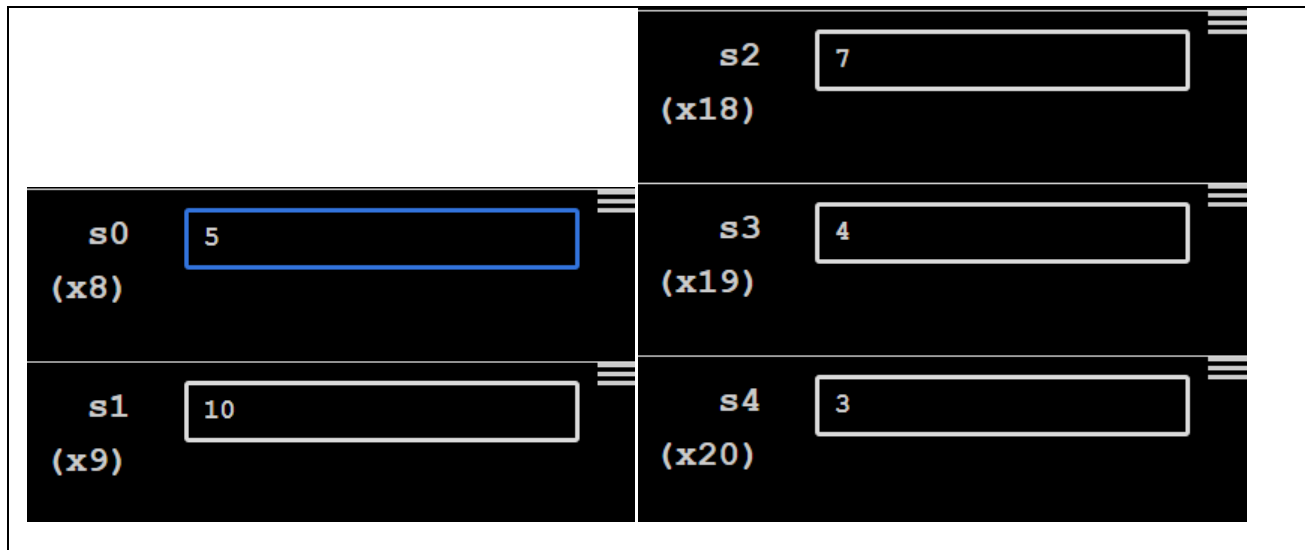
```

1  .data
2  .text
3  main:
4      li s0, 5      #a=5
5      li s1, 10     #b=10
6      li s3, 4      #d=4
7      li s4, 3      #e=3
8
9      beq s0, s1, true
10
11 else:
12
13      add s2, s3, s4 #c=d+e
14      j end
15
16 true:
17
18      sub s2, s3, s4 #c=d-e
19
20 end:

```

Output:

0x0	0x00500413	addi x8 x0 5	li s0, 5 #a=5
0x4	0x00A00493	addi x9 x0 10	li s1, 10 #b=10
0x8	0x00400993	addi x19 x0 4	li s3, 4 #d=4
0xc	0x00300A13	addi x20 x0 3	li s4, 3 #e=3
0x10	0x00940663	beq x8 x9 12	beq s0, s1, true
0x14	0x01498933	add x18 x19 x20	add s2, s3, s4 #c=d+e
0x18	0x0080006F	jal x0 8	j end
0x1c	0x41498933	sub x18 x19 x20	sub s2, s3, s4 #c=d-e



Task 5:

Description:

In this task we are required to shift a number i.e 2 so we firstly store number in s0 by adding it with x0 that is 0 then apply slli left shift that do $2 * 2^2$. The formula is $\text{number} * s^{\text{shifts}}$.

Code:

```
.data
.text
.globl main

    addi s0, x0, 2

    slli s2, s0, 2
```

```
.data
.text
.globl main
    addi s0, x0, 2
    slli s2, s0, 2
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00200413	addi x8 x0 2	addi s0, x0, 2
0x4	0x00241913	slli x18 x8 2	slli s2, s0, 2

s2
(x18)

8

Task 6:

Description:

In this task the c code is given using for condition so when we convert it into assembly language we initialize the values and load them to the registers and also the initialize condition like counter is 0 i=1 and till limit 5 then we start the loop in which we check the condition if i>5 so end other increment by one by using and immediate operation.

Code:

```
.data

.text

.globl main

    li s0, 0 #x0=0

    addi s1, x0, 1#i=1

    addi s2, x0, 5#limit 5

Loop:

    bgt s1, s2, end

    addi s0, s0, 1

    addi s1, s1, 1
```

j Loop

end:

```

1 .data
2 .text
3 .globl main
4     li s0, 0 #x0=0
5     addi s1, x0, 1#i=1
6     addi s2, x0, 5#limit 5
7 Loop:
8     bgt s1, s2, end
9     addi s0, s0, 1
10    addi s1, s1, 1
11    j Loop
12
13 end:
14
15
16

```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00000413	addi x8 x0 0	li s0, 0 #x0=0
0x4	0x00100493	addi x9 x0 1	addi s1, x0, 1#i=1
0x8	0x00500913	addi x18 x0 5	addi s2, x0, 5#limit 5
0xc	0x00994863	blt x18 x9 16	bgt s1, s2, end
0x10	0x00140413	addi x8 x8 1	addi s0, s0, 1
0x14	0x00148493	addi x9 x9 1	addi s1, s1, 1
0x18	0xFF5FF06F	jal x0 -12	j Loop

Task 7:

Description:

In this task the array of number is given number are 10 so firstly we allocate space for them $4 \times 10 = 40$ then we load the address of the number into register then initializes the variables i.e. $i=0$ and the limit till 5 by `addi` then starting the loop with the condition of $i > 5$ then end otherwise `addi` 5 and then store it to `s3` by using `slli` and `add`.

Code:

```
.data

    num: .space 40

.text

.globl main

main:

    la  s0, num

    addi s1, x0, 0

    addi s2, x0, 5

loop:

    bge s1, s2, end

    addi s3, s1, 5

    slli s4, s1, 2#

    add  s5, s0, s4

    sw  s3, 0(s5)
```

addi s1, s1, 1

j loop

end:

```
1 .data
2     num: .space 40
3
4 .text
5 .globl main
6 |
7 main:
8     la    s0, num
9     addi  s1, x0, 0
10    addi  s2, x0, 5
11
12 loop:
13    bge   s1, s2, end
14    addi  s3, s1, 5
15    slli  s4, s1, 2#
16    add   s5, s0, s4
17    sw    s3, 0(s5)
18    addi  s1, s1, 1
19    j     loop
20
21 end:
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x10000417	auipc x8 65536	la s0, num
0x4	0x00040413	addi x8 x8 0	la s0, num
0x8	0x00000493	addi x9 x0 0	addi s1, x0, 0
0xc	0x00500913	addi x18 x0 5	addi s2, x0, 5
0x10	0x0124DE63	bge x9 x18 28	bge s1, s2, end
0x14	0x00548993	addi x19 x9 5	addi s3, s1, 5
0x18	0x00249A13	slli x20 x9 2	slli s4, s1, 2#
0x1c	0x01440AB3	add x21 x8 x20	add s5, s0, s4
0x20	0x013AA023	sw x19 0(x21)	sw s3, 0(s5)
0x24	0x00148493	addi x9 x9 1	addi s1, s1, 1
0x28	0xFE9FF06F	jal x0 -24	j loop

s1 (x9)	5
------------	---

s2 (x18)	5
s3 (x19)	9
s4 (x20)	16
s5 (x21)	268435472

0x10000010	9	0	0	0
0x1000000C	8	0	0	0
0x10000008	7	0	0	0
0x10000004	6	0	0	0
0x10000000	5	0	0	0

Task 8:

Description:

In the task the c code is given and we have to write it to assembly language so firstly we loaded the initialize values in the register of counter and x then we started the outer loop where loaded the limit of x as 3 then we check the condition if $x > 3$ so move to next otherwise go to loop 2 where we initialize the limit y as 2 and check the condition if $y > 2$ so move to next and jump to loop1 otherwise perform addition and store new value in counter register .

Code:

```
.data

.text

.globl main

li s0, 0

li t0, 0
```

```
loop1:

    li  t2, 3

    bge t0, t2, end

    li  t1, 0

loop2:

    li  t3, 2

    bge t1, t3, next_x

    add s0, t0, t1

    addi t1, t1, 1

    j   loop2

next_x:

    addi t0, t0, 1

    j   loop1

end:
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00000413	addi x8 x0 0	li s0, 0
0x4	0x00000293	addi x5 x0 0	li t0, 0
0x8	0x00300393	addi x7 x0 3	li t2, 3
0xc	0x0272D263	bge x5 x7 36	bge t0, t2, end
0x10	0x00000313	addi x6 x0 0	li t1, 0
0x14	0x00200E13	addi x28 x0 2	li t3, 2
0x18	0x01C35863	bge x6 x28 16	bge t1, t3, next_x
0x1c	0x00628433	add x8 x5 x6	add s0, t0, t1
0x1c	0x00628433	add x8 x5 x6	add s0, t0, t1
0x20	0x00130313	addi x6 x6 1	addi t1, t1, 1
0x24	0xFF1FF06F	jal x0 -16	j loop2
0x28	0x00128293	addi x5 x5 1	addi t0, t0, 1
0x2c	0xFDFFF06F	jal x0 -36	j loop1
t0 (x5)	3		
t1 (x6)	2		
t2 (x7)	3		
s0 (x8)	3		

--